

Terceira Lista de Engenharia de Software RT

Capítulos 22 e 23 — Gerenciamento de Projetos de Software

Ian Sommerville — *Software Engineering*, 10^a ed. | Trabalho em grupo (4 componentes)

Capítulo 22 — Gerenciamento de Projetos

Questão 22.1

Enunciado

Explique por que a intangibilidade dos sistemas de software gera problemas especiais para gerenciamento de projetos de software.

Resposta

Ao contrário de sistemas físicos, o software não é visível: não é possível inspecionar um executável e avaliar seu progresso ou qualidade a olho nu. Isso cria três classes de problemas gerenciais:

1. **Dificuldade de medir progresso:** marcos tangíveis (como um componente mecânico entregue) não existem. Gerentes precisam confiar em métricas indiretas (linhas de código, pontos de função, testes passados), que são facilmente distorcidas. A síndrome “90% completo” é comum — o produto parece quase pronto por semanas.
2. **Invisibilidade de defeitos:** erros de lógica, condições de corrida e vulnerabilidades de segurança não se manifestam visualmente, podendo permanecer latentes até situações específicas de operação. Isso torna difícil garantir qualidade ao longo do processo.
3. **Gerenciamento de contratos e expectativas:** clientes não conseguem avaliar o que foi entregue sem executar o sistema. Isso complica a aceitação formal, o pagamento por milestones e a negociação de mudanças de escopo.
4. **Estimativas imprecisas:** sem componentes físicos de referência, estimar custo e prazo depende de experiência anterior com sistemas similares — dados raros em domínios novos.

Questão 22.2

Enunciado

Explique por que os melhores programadores nem sempre se tornam os melhores gerentes de software.

Resposta

Gerenciamento e programação são conjuntos de habilidades ortogonais. Conforme a Seção 22.1 de Sommerville, as principais atividades de um gerente

incluem: escrever propostas, planejamento, monitoração, relatórios e gerenciamento de pessoas. Programação exige raciocínio lógico profundo, concentração em problemas técnicos e trabalho predominantemente individual.

Conflitos específicos:

- **Comunicação interpessoal:** gerentes precisam negociar, motivar equipes e transmitir informações a públicos não técnicos. Bons programadores frequentemente preferem trabalho solitário e comunicação direta e técnica.
- **Tolerância à ambiguidade:** problemas gerenciais raramente têm uma solução única e correta. Programadores costumam ser treinados a buscar soluções ótimas, o que pode paralisar decisões gerenciais necessariamente baseadas em informação incompleta.
- **Perda de contribuição técnica:** gerentes deixam de codificar. Programadores excepcionais podem perceber a promoção como perda de produtividade e satisfação pessoal.
- **Responsabilidade difusa:** o sucesso de um time não é atribuível a um único indivíduo, o que contrasta com o reconhecimento direto que bons programadores recebem por soluções técnicas elegantes.

Questão 22.3

Enunciado

Usando instâncias de problemas de projetos relatados na literatura, liste as dificuldades e os erros de gerenciamento que ocorreram em projetos de programação que falharam.

Resposta

Com base em *The Mythical Man-Month* (Brooks, 1975) e outros casos:

- **Lei de Brooks (IBM OS/360):** adicionar pessoal a um projeto atrasado atrasa-o ainda mais. O overhead de comunicação cresce em $O(n^2)$ com o número de pessoas. A IBM estimou errado a complexidade do OS/360 e, ao adicionar equipes, multiplicou os atrasos.
- **Estimativa otimista (segundo sistema de Brooks):** o segundo sistema de um desenvolvedor tende ao excesso de complexidade e funcionalidades. O OS/360 sofreu disso — funcionalidades desnecessárias foram adicionadas, aumentando a complexidade sem valor correspondente.
- **Denver Airport Baggage System (1994):** \$193 milhões em prejuízo e 16 meses de atraso por subestimação da complexidade de integração, falta de prototipação e pressões políticas sobre o cronograma.
- **Healthcare.gov (2013):** falha no lançamento do portal do governo americano por gestão fragmentada entre múltiplos contratantes, falta de um arquiteto-chefe e testes insuficientes de integração até semanas antes do lançamento.

- **Erros recorrentes identificados:** cronogramas não realistas sob pressão comercial; ausência de controle de mudanças formal; falta de comunicação entre subequipes; testes relegados ao final do projeto; mitos de produtividade (“pessoas-mês” como unidade de medida de capacidade).

Questão 22.4

Enunciado

Além dos riscos mostrados na Tabela 22.1, identifique pelo menos seis outros possíveis riscos que podem surgir em projetos de software.

Resposta

1. **Risco de segurança cibernética:** vulnerabilidades descobertas em componentes de terceiros (ex.: Log4Shell) podem invalidar decisões arquiteturais tardias no projeto.
2. **Risco de integração de sistemas legados:** interfaces não documentadas ou indisponíveis de sistemas legados podem impedir a migração ou integração planejada.
3. **Risco de rotatividade de pessoal-chave:** saída de membros com conhecimento crítico não documentado pode paralisar o projeto.
4. **Risco regulatório:** mudanças em legislação (ex.: LGPD, GDPR) durante o desenvolvimento podem exigir retrabalho arquitetural significativo.
5. **Risco de fornecedor/terceiro:** falência ou mudança de estratégia de um fornecedor de componente crítico (ex.: descontinuação de API) pode inviabilizar partes do sistema.
6. **Risco de desempenho:** o sistema atende aos requisitos funcionais mas não às metas de desempenho sob carga real, exigindo refatoração extensiva.
7. **Risco de adoção:** usuários resistem ao novo sistema por questões de usabilidade ou mudança de processos, reduzindo o retorno sobre o investimento.

Questão 22.5

Enunciado

Contratos de preço fixo podem mover o risco de projeto para o contratante. Sugira como o uso desses contratos pode aumentar a probabilidade de surgirem riscos de produtos.

Resposta

Em contratos de preço fixo, o contratante (desenvolvedor) absorve qualquer custo acima do valor contratado. Para proteger sua margem, o contratante tende a:

- **Cortar escopo de qualidade:** reduzir testes, revisões de código e do-

cumentação — atividades que não são visíveis ao cliente mas são críticas para a confiabilidade do produto final.

- **Aceitar requisitos ambíguos sem negociá-los:** cada esclarecimento posterior pode ser cobrado como mudança, incentivando a entrega de um produto que atende à letra, mas não ao espírito dos requisitos.
- **Usar tecnologias ou arquiteturas de menor custo de desenvolvimento:** que podem não ser as mais adequadas para manutenibilidade ou desempenho a longo prazo.
- **Subestimar deliberadamente:** para ganhar o contrato, contratantes podem oferecer lances inviáveis, sabendo que a recuperação virá por meio de cobranças de mudanças ou entregando um produto de qualidade inferior.

Todos esses comportamentos aumentam o risco de produto: o sistema entregue pode ser frágil, inseguro ou não atender às necessidades reais, mesmo cumprindo formalmente o contrato.

Questão 22.6

Enunciado

Explique por que manter todos os membros de um grupo informados sobre progressos e decisões técnicas de um projeto pode melhorar a coesão de grupo.

Resposta

A coesão de grupo é fortalecida quando todos os membros compartilham um modelo mental comum do projeto. Informar progressos e decisões técnicas:

1. **Gera senso de pertencimento:** membros informados sentem que contribuem para um objetivo compartilhado, não apenas para sua tarefa isolada. Isso reduz o comportamento de “silo”.
2. **Cria responsabilidade coletiva:** quando todos sabem o estado do projeto, erros e atrasos são detectados mais cedo e a equipe age de forma proativa.
3. **Facilita decisões distribuídas:** membros informados podem tomar micro-decisões alinhadas às decisões estratégicas sem precisar escalar cada questão.
4. **Reduz rumores e ansiedade:** transparência sobre dificuldades e riscos evita o “jogo de telefone” que frequentemente causa conflitos e desmotivação.
5. **Promove aprendizado coletivo:** decisões técnicas explicadas à equipe aumentam a competência coletiva e reduzem a dependência de especialistas individuais.

Questão 22.7

Enunciado

Quais os problemas que podem surgir nas equipes de Extreme Programming em que muitas decisões de gerenciamento são atribuídas aos membros de equipe?

Resposta

- **Conflitos não resolvidos:** sem uma autoridade clara, divergências técnicas (ex.: escolha de arquitetura, prioridade de histórias) podem se arrastar sem decisão, criando bloqueios.
- **Planejamento inconsistente:** diferentes membros podem ter visões conflitantes sobre velocidade de desenvolvimento e estimativas de story points, dificultando o planejamento de releases.
- **Falta de accountability:** quando decisões são coletivas, ninguém é responsável pelos resultados. Erros gerenciais podem não ser reconhecidos ou corrigidos.
- **Pressão grupal:** membros menos assertivos podem silenciar diante de colegas mais dominantes, perdendo-se perspectivas valiosas.
- **Dificuldade com stakeholders externos:** clientes e gestores precisam de um ponto de contato único. Uma equipe sem hierarquia clara pode comunicar mensagens inconsistentes.
- **Escalabilidade:** o modelo funciona bem em equipes pequenas (até 10 pessoas). Em projetos maiores, a autogestão coletiva torna-se caótica sem mecanismos formais de coordenação.

Questão 22.8

Enunciado

Escreva um estudo de caso, no estilo usado neste capítulo, para ilustrar a importância da comunicação em uma equipe de projeto com membros remotos.

Resposta

Caso: Sistema de Monitoramento de Infraestrutura Hospitalar

Uma equipe de 8 desenvolvedores distribuídos em Manaus, São Paulo e Lisboa foi contratada para desenvolver um sistema de monitoramento de leitos hospitalares em 9 meses. O time de Manaus era responsável pelo backend (sensores IoT), o time de São Paulo pelo frontend web e o time de Lisboa pela integração com o sistema de prontuários.

Problema: Os times de Manaus e Lisboa definiram, independentemente, formatos de mensagem JSON diferentes para os eventos de sensor. A discrepância só foi descoberta na integração (mês 7), quando o frontend exibia dados incorretos. A correção exigiu 3 semanas adicionais e refatoração do módulo de persistência.

Causa raiz: As reuniões semanais de sincronização focavam em progresso funcional individual, sem revisão de contratos de API. Não havia um repositório

central de especificações de interface revisado por todos os times. As diferenças de fuso horário (Manaus–Lisboa: -4h) dificultavam comunicação assíncrona e resolução rápida de dúvidas.

Lição: Equipes distribuídas necessitam de um contrato de interface explícito (ex.: OpenAPI), revisado e aprovado por todos os times antes do início da implementação. Reuniões de sincronização devem incluir revisão de decisões técnicas de impacto transversal, não apenas relatórios de progresso. Uma ferramenta de comunicação com canais por componente e responsável único por decisões técnicas teria prevenido o problema.

Questão 22.9

Enunciado

Seu gerente solicitou que você entregue o software dentro do cronograma, o qual só poderá ser cumprido caso você peça à sua equipe que trabalhe horas extras não remuneradas. Todos os membros têm crianças pequenas. Discuta se você deve aceitar essa demanda ou convencer sua equipe a dar seu tempo à organização.

Resposta

Fatores a considerar:

- **Ético-profissional:** o Código de Ética da ACM e da IEEE-CS exige que engenheiros tratem colegas com justiça e não permitam que pressões comerciais comprometam o bem-estar da equipe.
- **Responsabilidade parental:** solicitar horas extras a membros com filhos pequenos impõe custo social real às suas famílias, que não têm voz na negociação contratual.
- **Qualidade do software:** equipes exaustas cometem mais erros. Horas extras não remuneradas frequentemente resultam em dívida técnica que, a longo prazo, atrasa projetos futuros.
- **Comprometimento contratual:** se o cronograma foi estabelecido com estimativas ruins, o problema é de planejamento — não de esforço da equipe.

Decisão recomendada: Não aceitar a demanda silenciosamente. A postura ética é comunicar ao gerente o impacto humano e técnico da solicitação, propor renegociação do escopo ou do prazo com o cliente, e documentar formalmente as consequências da decisão. Se a demanda for imposta, comunicá-la à equipe de forma transparente — sem coerção velada — e deixar que cada membro decida individualmente se pode ou não contribuir com tempo adicional voluntário. Jamais pressionar quem recusar.

Questão 22.10

Enunciado

Como programador, você tem a possibilidade de ser promovido para gerente de projetos, mas sente que pode contribuir mais eficazmente como técnico. Discuta se você deve aceitar a promoção.

Resposta

Argumentos para recusar:

- Contribuição técnica de alta qualidade tem impacto direto e mensurável em sistemas críticos; gerenciamento ruim pode neutralizar esse impacto.
- Satisfação pessoal e motivação intrínseca são preditores fortes de produtividade a longo prazo. Assumir um papel que não mobiliza as competências valorizadas pelo profissional tende a gerar frustração.
- Muitas organizações precisam mais de engenheiros sêniores do que de gerentes adicionais.

Argumentos para aceitar:

- Gerentes com sólida base técnica tomam decisões melhores e ganham credibilidade da equipe. A organização pode se beneficiar mais de alguém que entende os detalhes técnicos do que de um gerente puramente administrativo.
- Exposição gerencial pode revelar aptidões não descobertas e abrir novas perspectivas de carreira.

Recomendação: negociar com a organização alternativas como papel de *Tech Lead* — responsabilidade técnica e alguma responsabilidade de coordenação, sem abandono da contribuição técnica. Se a promoção for aceita, deve-se fazê-lo com comprometimento genuíno e plano de desenvolvimento das habilidades gerenciais. Aceitar a promoção sem motivação apenas para progressão salarial é prejudicial ao profissional, à equipe e ao projeto.

Capítulo 23 — Planejamento de Projetos

Questão 23.1

Enunciado

Em quais circunstâncias uma empresa poderia, justificadamente, cobrar um preço muito mais elevado do que o custo estimado de software, além de uma margem de lucro razoável?

Resposta

1. **Monopólio de conhecimento específico:** se a empresa é a única com expertise no domínio do cliente (ex.: sistema de defesa ou industrial específico), pode precificar o valor percebido, não o custo.
2. **Risco assumido pelo fornecedor:** em contratos de preço fixo com re-

- quisitos altamente incertos, o prêmio de risco justifica uma margem maior.
3. **Software crítico de segurança:** sistemas médicos, aviônicos e nucleares exigem certificação formal, documentação extensiva e processos de garantia de qualidade caros que elevam os custos reais.
 4. **Entrega urgente:** prazos muito comprimidos exigem horas extras, contratação rápida ou interrupção de outros projetos, justificando preço premium.
 5. **Propriedade intelectual:** quando o software incorpora algoritmos proprietários ou pesquisa avançada, o valor intelectual supera o custo de desenvolvimento.

Questão 23.2

Enunciado

Explique por que o processo de planejamento de projeto é iterativo e por que um plano deve ser continuamente revisto durante um projeto de software.

Resposta

O planejamento é iterativo porque as premissas iniciais são inevitavelmente incompletas ou incorretas:

- **Incerteza inicial:** no início do projeto, requisitos são vagos, tecnologias não testadas e a capacidade da equipe é incerta. Estimativas baseadas nessas premissas degradam-se rapidamente.
- **Descobertas durante o desenvolvimento:** cada fase revela complexidades não previstas. Um módulo estimado em 5 dias pode levar 15 após análise detalhada.
- **Mudanças externas:** requisitos do cliente mudam; membros de equipe entram ou saem; tecnologias de terceiros mudam de versão.
- **Feedback de milestones:** revisões periódicas revelam desvios entre o planejado e o executado, exigindo replanejamento.

Um plano não revisado torna-se desatualizado e perde seu valor como instrumento de controle. O plano iterativo é mais eficaz porque incorpora aprendizado contínuo e mantém as estimativas ancoradas na realidade corrente do projeto.

Questão 23.3

Enunciado

Explique rapidamente a finalidade de cada uma das seções em um plano de projeto de software.

Resposta

Seção	Finalidade
Introdução	Define escopo, objetivos e contexto; estabelece quem vai ler o documento e o que se espera do projeto.
Organização do projeto	Descreve a estrutura da equipe, papéis e responsabilidades, e a relação com o cliente.
Análise de riscos	Identifica riscos, sua probabilidade e impacto, e define estratégias de mitigação e contingência.
Req. de recursos de hardware e software	Lista ferramentas, infraestrutura e ambientes necessários para desenvolvimento e operação.
Divisão do trabalho	Decompõe o projeto em atividades, atribui responsáveis e define dependências (WBS).
Cronograma do projeto	Presenta o sequenciamento temporal das atividades (Gantt, CPM), marcos e datas de entrega.
Mecanismos de monitoramento e relatórios	Define como o progresso será medido, com que frequência e em que formato relatórios serão produzidos.

Questão 23.4

Enunciado

Sugira quatro maneiras de reduzir o risco em uma estimativa de custo.

Resposta

1. **Usar múltiplas técnicas de estimativa:** combinar COCOMO, pontos de função e julgamento especializado. Convergência entre os métodos aumenta a confiança; divergência sinaliza incerteza que deve ser explicitada como risco.
2. **Estimativas incrementais (top-down + bottom-up):** decompor o sistema em componentes e estimar cada um individualmente (bottom-up), comparando com uma estimativa global (top-down). Discrepâncias revelam itens omitidos.
3. **Usar dados históricos de projetos similares:** repositórios organizacionais de projetos passados (com tamanho, custo real e desvios) fornecem base empírica para calibrar estimativas.
4. **Inclusão explícita de reservas de contingência:** reservas para risco (ex.: 20–30% do esforço estimado) transformam a incerteza de uma ameaça implícita em um buffer gerenciado e visível para o cliente.

Questão 23.5 — Gráfico de Barras da Tabela 23.7

Enunciado

A Tabela 23.7 define tarefas com durações e dependências. Desenhe um gráfico de barras mostrando o cronograma de projeto.

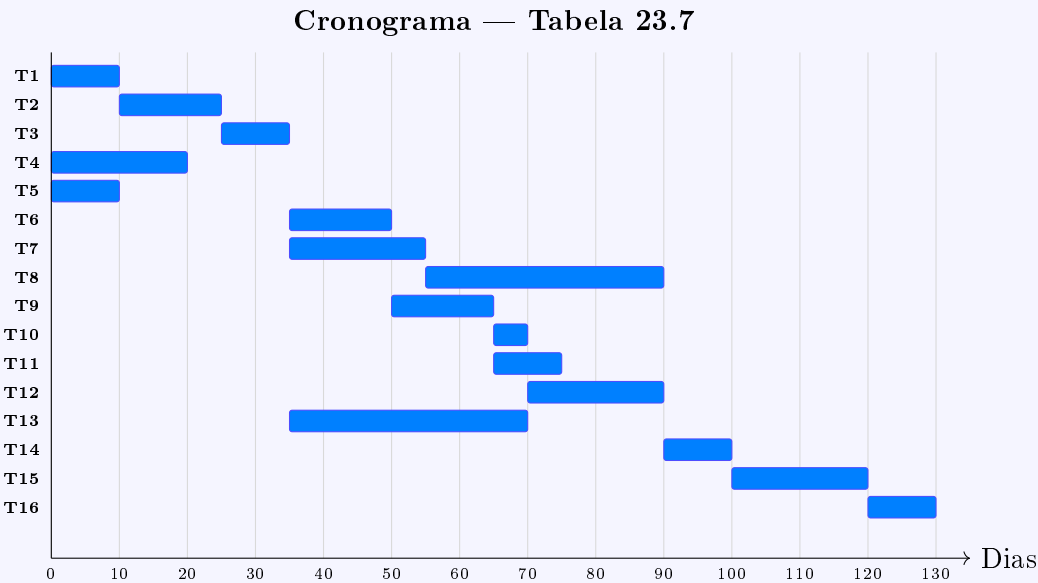
Resposta

Cronograma calculado (início mais cedo para cada tarefa):

Tarefa	ES	EF	Dependências
T1	0	10	—
T2	10	25	T1
T3	25	35	T1, T2
T4	0	20	—
T5	0	10	—
T6	35	50	T3, T4
T7	35	55	T3
T8	55	90	T7
T9	50	65	T6
T10	65	70	T5, T9
T11	65	75	T9
T12	70	90	T10
T13	35	70	T3, T4
T14	90	100	T8, T9
T15	100	120	T2, T14
T16	120	130	T15

Duração total: 130 dias

Gráfico de Gantt (escala: 1 unidade = 10 dias):



Questão 23.6 — Reorganização com $T5 = 40$ dias

Enunciado

Suponha que ocorra um retrocesso grave e, em vez de levar 10 dias, a tarefa T5 leva 40 dias. Elabore novos gráficos de barras, mostrando como o projeto pode ser reorganizado.

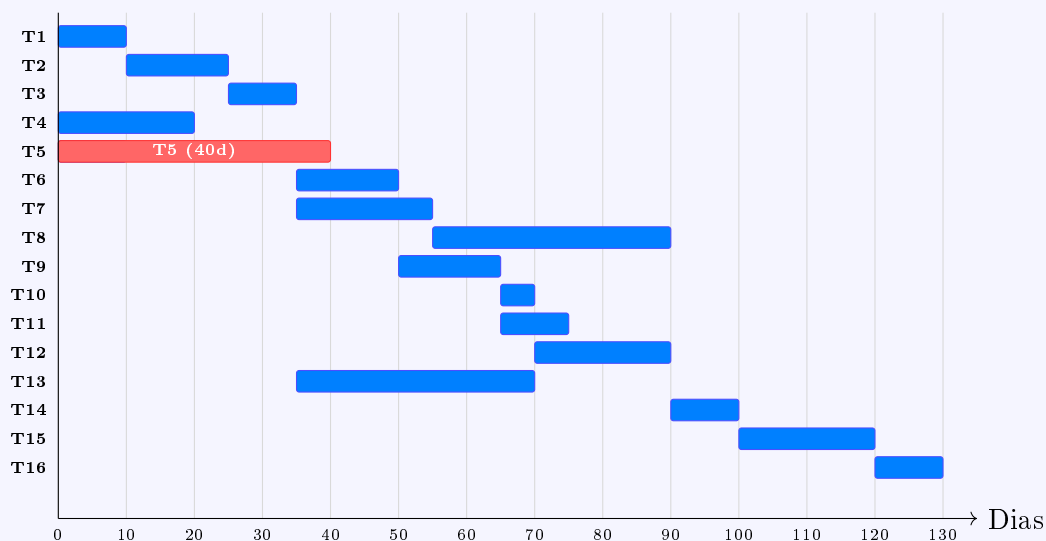
Resposta

Com $T5 = 40$ dias:

- T5: EF passa de 10 para **40 dias**.
- T10 depende de T5 e T9. Com T9 terminando no dia 65, a restrição de T10 continua sendo T9: $ES(T10) = \max(40, 65) = 65$.
- **Nenhuma outra tarefa é afetada** — T5 não é caminho crítico.
- **Duração total do projeto permanece 130 dias**.

Impacto: apenas o comprimento da barra de T5 no Gantt aumenta. A reorganização consiste em garantir que os recursos alocados a T5 sejam mantidos por 40 dias em vez de 10, sem impacto nas demais atividades.

Cronograma Reorganizado ($T5=40$ dias)



Questão 23.7

Enunciado

Explique os potenciais problemas com o jogo de planejamento do XP quando o software tem altos requisitos de desempenho ou de confiança.

Resposta

O jogo de planejamento do XP baseia-se em histórias de usuário priorizadas pelo cliente por valor de negócio. Problemas em sistemas de alto desempenho ou alta confiança:

- **Requisitos não-funcionais invisíveis nas histórias:** desempenho, segurança e confiabilidade raramente aparecem como histórias de usuário discretas. São propriedades emergentes da arquitetura e difíceis de priorizar por valor de negócio percebido.
- **Decisões arquiteturais adiadas:** o XP encoraja o design emergente (YAGNI), mas sistemas de alta confiança precisam de uma arquitetura de segurança definida cedo — mudar tarde é extremamente custoso.
- **Ausência de análise formal de riscos:** histórias de segurança crítica (ex.: “o sistema não deve emitir dose de radiação errada”) não são prioritáveis apenas por valor de negócio; são obrigações legais.
- **Ciclos curtos vs. certificação:** certificações de sistemas críticos (DO-178C, IEC 61508) exigem documentação de processo pesada, incompatível com os ciclos semanais e baixa cerimônia do XP.
- **Refatoração não é livre:** em sistemas de alto desempenho, refatorar uma estrutura de dados pode quebrar garantias de tempo de resposta garantidas previamente.

Questão 23.8

Enunciado

Sistema de radioterapia com estimativa COCOMO de 26 pessoas-mês, com todos os multiplicadores de driver de custo definidos como 1. Explique por que essa estimativa deve ser ajustada, cite quatro fatores e proponha valores.

Resposta

Definir todos os multiplicadores como 1 assume que o sistema é *médio* em todas as dimensões — o que é claramente inadequado para um sistema médico crítico embarcado. Os fatores COCOMO II que precisam de ajuste:

Fator	Justificativa	Valor	Multiplicador
RELY (Confiabilidade)	Sistema de radioterapia — falha pode resultar em morte do paciente. Nível “extra alto”.	Extra alto	1,39
CPLX (Complexidade)	Controle em tempo real de hardware específico (processador dedicado, 256 MB RAM fixos), comunicação com BD hospitalar, interface crítica de segurança. Nível “muito alto”.	Muito alto	1,30
STOR (Restrição de armazenamento)	Execução em memória fixa de 256 MB com código de controle em tempo real. Restrição “muito alta” (uso acima de 70% da memória).	Muito alto	1,21
SECU (Segurança)	Restrição de segurança funcional (requisito de integridade de segurança nível SIL 3 ou 4). Exige análise formal, FMEA e rastreabilidade completa.	Extra alto	1,30
Produto dos multiplicadores: $1,39 \times 1,30 \times 1,21 \times 1,30 \approx 2,84$ Estimativa ajustada: $26 \times 2,84 \approx 74$ pessoas-mês.			

Questão 23.9

Enunciado

Explique por que os modelos de estimativa de esforço, como o COCOMO, podem não funcionar bem quando aplicados a sistemas muito grandes.

Resposta

1. **Dados históricos escassos:** COCOMO foi calibrado com projetos de até algumas centenas de KLOC. Sistemas com milhões de LOC são raros e têm características únicas não capturadas na base de dados histórica.
2. **Não-linearidade da complexidade:** o expoente de escala (E) em $PM = A \times (KLOC)^E$ assume que a complexidade cresce de forma regular com o tamanho. Em sistemas muito grandes, a complexidade de integração, testes de sistema e comunicação organizacional cresce de forma super-linear, além do que o expoente modela.
3. **Divisão em subsistemas:** sistemas muito grandes são desenvolvidos por múltiplas equipes e organizações em paralelo, com interfaces complexas. O COCOMO não modela adequadamente o custo de integração entre grandes subsistemas.
4. **Evolução durante o desenvolvimento:** em projetos de vários anos, os requisitos mudam significativamente. O LOC estimado no início é uma

meta-alvo instável, invalidando as estimativas baseadas nele.

Questão 23.10

Enunciado

É ético para uma empresa cotar um preço baixo para um contrato de software sabendo que os requisitos são ambíguos e que eles poderão cobrar um preço elevado para mudanças posteriores solicitadas pelo cliente?

Resposta

Não, esta prática é eticamente problemática pelas seguintes razões:

- **Enganosidade intencional:** a empresa conhece a assimetria de informação (que os requisitos são ambíguos e gerarão mudanças custosas) e a explora deliberadamente. Isso viola o princípio ético de agir de boa-fé com o cliente.
- **Dano ao cliente:** o cliente toma sua decisão de compra baseado em um preço que não reflete o custo real. O orçamento será excedido, potencialmente comprometendo outros projetos ou a viabilidade financeira da organização compradora.
- **Conflito com códigos de ética profissional:** o Código de Ética da ACM (Princípio 1.3) determina que profissionais de computação devem ser honestos e não enganar clientes ou empregadores.
- **Efeito sistêmico:** a prática destrói a confiança no setor, eleva custos de contratação (clientes precisam de auditorias extensas e contratos mais complexos) e prejudica empresas éticas que propõem preços justos e perdem contratos para a concorrência desonesta.

A alternativa ética é: identificar e documentar a ambiguidade dos requisitos antes da proposta, cotar um preço que inclua uma reserva de risco explícita, e propor uma fase de detalhamento de requisitos paga antes da estimativa final.

Exercício 12.1 — Estimativa de Custo com COCOMO

Enunciado

Sistema GUI-based com 75.000 LOC (complexo). Calcule: (a) esforço total em PM; (b) distribuição por fase; (c) esforço e custos por atividade em cada fase; (d) custo total do projeto.

(a) **Esforço total:**

$$PM = 3,0 \times (KLOC)^{1,12} = 3,0 \times 75^{1,12} = 3,0 \times 125,91 \approx \boxed{377,7 \text{ PM}}$$

(b) **Distribuição por fase:**

Fase	%	PM
Requirements Analysis & Conception	8%	30,2 PM
Design	18%	68,0 PM
Implementation	48%	181,3 PM
Integration & Test	26%	98,2 PM
Total	100%	377,7 PM

(c) Esforço e custos por atividade — Fase: Requirements Analysis & Conception (30,2 PM):

Atividade	%	PM	Taxa (€/PM)	Custo (€)
Requirements Analysis	42%	12,69	10.000	126.920
Product Design	16%	4,84	6.000	29.010
Programming	10%	3,02	6.000	18.130
Plan Tests	6%	1,81	9.000	16.320
Verification & Validation	10%	3,02	9.000	27.200
Project Management	8%	2,42	12.000	29.010
Configuration Mgmt	3%	0,91	7.000	6.350
Quality Assurance	5%	1,51	7.000	10.580
Total RAC	100%	30,22	—	263.520

Os valores para Design (506.550 €), Implementation (1.292.770 €) e Integration & Test (720.870 €) foram computados analogamente usando as colunas correspondentes da Figura 1.

(d) Custo total do projeto:

Fase	Custo (€)
Requirements Analysis & Conception	263.520
Design	506.550
Implementation	1.292.770
Integration & Test	720.870
TOTAL	2.783.710

Exercício 12.2 — Rede de Planejamento (CPM / MPM)

Enunciado

Para o depósito automatizado, construa: (a) rede CPM com caminho crítico; (b) rede MPM; (c) gráfico de barras (Gantt).

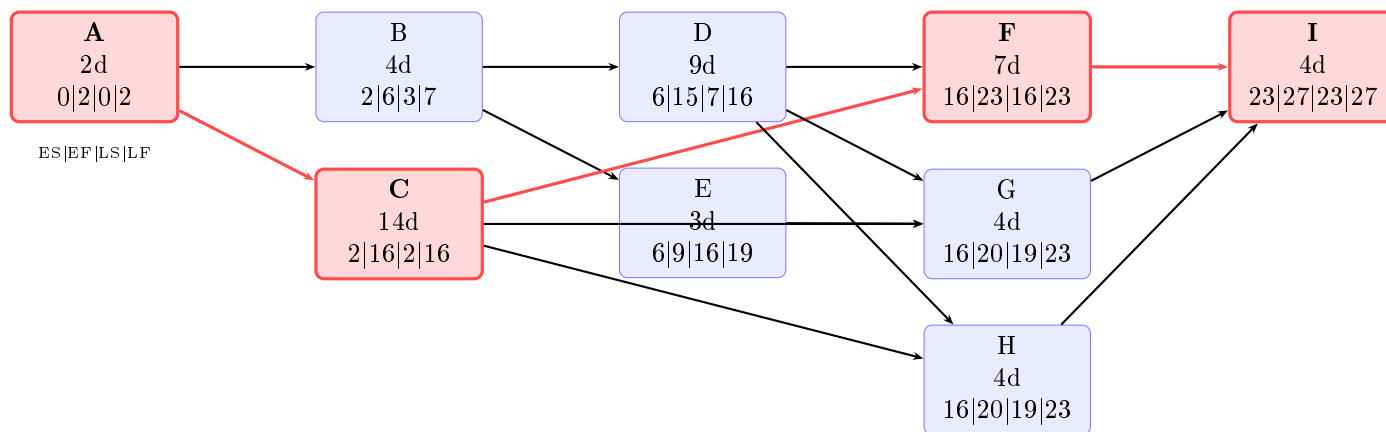
(a) Rede CPM — Análise de Caminhos Críticos

Cálculo dos tempos mais cedo e mais tarde:

Atividade	Dur.	ES	EF	LS	LF	Folga
A (Preparação)	2	0	2	0	2	0 *
B (Projeto depósito)	4	2	6	3	7	1
C (Construção)	14	2	16	2	16	0 *
D (Desenv. sist. controle)	9	6	15	7	16	1
E (Provisão mecânica)	3	6	9	16	19	10
F (Teste componentes edif.)	7	16	23	16	23	0 *
G (Teste componentes mec.)	4	16	20	19	23	3
H (Instalação edif.)	4	16	20	19	23	3
I (Integração e introdução)	4	23	27	23	27	0 *

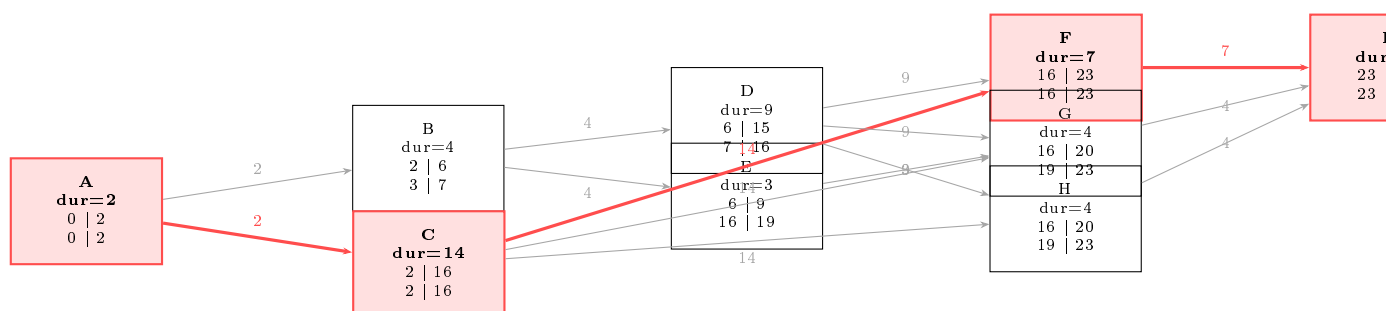
Caminho crítico: A → C → F → I (duração: 2 + 14 + 7 + 4 = **27 dias**)

Diagrama CPM (Activity-on-Arrow):

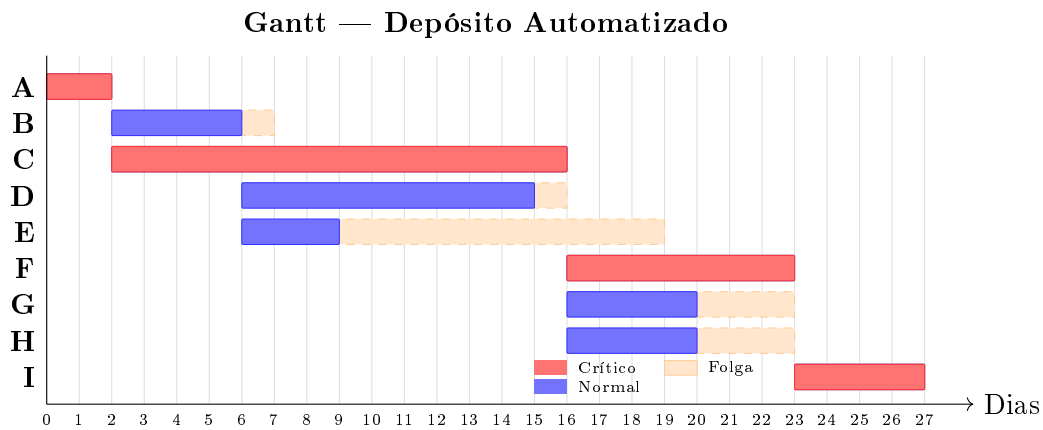


(b) Rede MPM (Metra Potential Method / Activity-on-Arrow):

No MPM, os nós são as atividades e as arestas são ponderadas pela duração do predecessor (ou pela relação temporal explícita). Cada nó mostra: ES | EF na linha superior, LS | LF na linha inferior, com a duração ao centro.



(c) Gráfico de Barras (Gantt) para o Depósito Automatizado:



Exercício 12.3 — Net Plans vs. Diagramas de Gantt

Enunciado

Nomeie as vantagens e desvantagens de net plans e diagramas de Gantt.

Resposta

	Net Plan (CPM/MPM)	Diagrama de Gantt
Vantagens	• Representa explicitamente dependências entre atividades • Identifica o caminho crítico e as folgas de cada atividade • Permite análise formal do impacto de atrasos (“what-if”) • Útil para projetos complexos com muitas interdependências	• Visualização intuitiva do cronograma temporal • Fácil de ler por não-especialistas e clientes • Mostra claramente quem faz o quê e quando • Ferramentas amplamente disponíveis (MS Project, etc.)
Desvantagens	• Difícil de ler para não-especialistas • Não mostra diretamente alocação de recursos • Construção e manutenção manual são trabalhosas • Para projetos muito grandes, torna-se visualmente incompreensível	• Não representa explicitamente as dependências • Não identifica o caminho crítico automaticamente • Atualizações frequentes exigem redesenho • Menos adequado para análise formal de riscos de prazo

Conclusão: Projetos complexos se beneficiam de ambas as ferramentas com-

plementarmente: a rede (CPM/MPM) para análise e planejamento técnico, e o Gantt para comunicação com a equipe e stakeholders.

Referência: Ian Sommerville. **Software Engineering**, 10^a ed. Pearson, 2016. Capítulos 22 (Gerenciamento de Projetos) e 23 (Planejamento de Projetos).